

User-defined erroneous behaviour

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Motivation](#)
4. [Proposal: `std::erroneous`](#)
5. [Impact and implementability](#)
6. [Proposed wording](#)
7. [Acknowledgements](#)
8. [References](#)

Revision history

- P3232R0: Initial revision.

Summary

We propose a language-support library function that has no effect other than to cause erroneous behaviour. This allows user-defined APIs to include erroneous behaviour.

Motivation

The purpose of erroneous behaviour, introduced in [P2795R5](#), is to provide well-defined behaviour in the presence of certain programming errors, thereby mitigating the safety and security implications of said errors: Erroneous behaviour is part of the observable behaviour of a program, and the compiler must not assume that it does not happen (unlike undefined behaviour).

Whereas P2795R5 only prescribed erroneous behaviour for a particular operation (namely reading an uninitialized variable with automatic storage duration), being able to declare parts of an API erroneous is

also useful for user-defined APIs that have preconditions (i.e. a “narrow contract”). It is a programming error to invoke such an API when preconditions are not met, but currently, attempts to make this API “safe”, that is, to limit the damage caused by calling it out of contract, suffer various shortcomings:

- We can leave the behaviour out of contract undefined. This is easy to specify and follows the Standard Library convention, but has poor safety implications (as basically anything could happen).
- We can fully specify the behaviour out of contract (e.g. terminating or throwing an exception). This makes the API technically have a wide contract (as there are no technical preconditions), and the intended preconditions have to be communicated out-of-band, e.g. just in natural language.
- We can check the preconditions in a subset of cases (e.g. with `assert` when `NDEBUG` is not defined. This retains the narrow contract, but leaves the safety implications somewhat opaque.
- A hypothetical "contracts" facility in the language (e.g. see [P2900R6](#) for a recent iteration) would allow a programmatic statement of the precondition, but there is not yet an agreed-upon definition of the precise semantics.

To illustrate this on a simple example, let us consider a small function that computes the quotient of two floating point numbers and has the precondition that the denominator not be zero:

Undefined behaviour

```
// Precondition: `den` must not be zero
float quotient(float num, float den) {
    return num / den;
}
```

Checked with `assert`

```
// Precondition: `den` must not be zero
float quotient(float num, float den) {
    assert(den != 0);
    return num / den;
}
```

Well-defined violation

```
// Precondition: `den` must not be zero,
// terminates otherwise
float quotient(float num, float den) {
    if (den == 0) { std::abort(); }
    return num / den;
}
```

**With a “contracts”
facility,
precondition:**

```
// Precondition and contract: `den` must  
not be zero.  
float quotient(float num, float den)  
pre(den != 0) {  
    // Option #1: undefined behaviour on  
violation  
    /* nothing */  
  
    // Option #2: well-defined behaviour on  
violation  
    if (den == 0) { return -2; }  
  
    return num / den;  
}
```

**With a “contracts”
facility,
contract_assert:**

```
// Precondition: `den` must not be zero.  
float quotient(float num, float den) {  
    contract_assert(den != 0);  
  
    // Option #1: undefined behaviour on  
violation  
    /* nothing */  
  
    // Option #2: well-defined behaviour on  
violation  
    if (den == 0) { return -2; }  
  
    return num / den;  
}
```

**Proposal: violation is
erroneous**

```
// Returns the quotient `num`/`den`;  
// if `den` is zero, returns -2  
erroneously.  
float quotient(float num, float den) {  
    if (den == 0) { std::erroneous();  
return -2; }  
    return quotient(unsafe_unchecked, num,  
den);  
}
```

```

}

// As above, but precondition violation
is undefined
float quotient(unsafe_unchecked_t, float
num, float den) {
    return num / den;
}

```

The final row demonstrates the proposed feature: by allowing user-defined code to be erroneous, we can offer a safe-by-default API that has just the same preconditions as an unsafe API would have had, but with well-defined behaviour (or termination; see P2795R5) in case the user makes a mistake. The original, unchecked API can be provided as a separate, explicitly annotated overload.

Proposal: `std::erroneous`

We propose a language-support function `std::erroneous` that has no effect other than to have erroneous behaviour.

The proposed function is to `std::unreachable` as erroneous behaviour is to undefined behaviour:

Function	Behaviour if invoked
<code>std::unreachable</code>	undefined behaviour
<code>std::erroneous</code>	erroneous behaviour; no effect

Impact and implementability

Platforms that diagnose erroneous behaviour will presumably provide some builtin hook with which the feature can be implemented. Otherwise, it is always conforming to implement this feature as a no-op.

Alternatives

An `[[erroneous]]` attribute was suggested during informal discussions, but it does not seem compelling: For example, it does not seem useful to annotate an entire function as always having erroneous behaviour. Instead, it is more composable to express this concern separately and once and for all, namely in the proposed function `std::erroneous`.

Proposed wording

In either [support] or [diagnostics], in a header to be determined, add a new function:

```
// User-defined erroneous behavior  
void erroneous();
```

Add the specification:

User-defined erroneous behavior

```
void erroneous();
```

Effects: The behavior is erroneous; calling this function has no effect otherwise.

Acknowledgements

Many thanks to Barry Revzin for helpful questions and discussion, and to members of SG21 for feedback on the connection to contracts.

References

- Thomas Köppe, [P2795R5](#): *Erroneous behaviour for uninitialized reads*.
- Thomas Köppe, [N4981](#): *Working Draft, Programming Languages — C++*.
- Joshua Berne et al., [P2900R6](#): *Contracts for C++*.